Multi-LLM Chat Client – OpenAI, DeepSeek, Google Gemini, Hugging Face & GroqCloud

---

Table of Contents

Overview

Architecture

1. Client Layer

2. Server Layer

3. Data Storage

4. Helper Scripts

Unique Context Management

Features in Detail

Chat Interface

Streaming Responses

OpenAI Integration

Google Gemini Integration

Hugging Face Integration

GroqCloud Integration

LLM Settings Panel

429 Rate Limit Handling

Clipboard Handler (Ctrl+V)

File Upload with Security Check

Umlaut Placeholder System

DeepThink Mode

Model Detection & Capabilities

Vision (Image) Support

Multi-Language System

Kompressor — Intelligent Context Compression

Kompressor is Headroom's context compression engine. It automatically detects large tool outputs, file reads, and search results in your conversation history and compresses them before they reach the LLM — saving tokens and money without losing the information the model needs.

The implementation lives in `headroom/transforms/kompressor.py`.

Core Concept

Every message in the conversation is scanned. If a message's **estimated token count** exceeds the configured threshold, Kompressor compresses it. The compressed version replaces the original in the message list sent to the LLM. The original is never modified — Kompressor operates on a copy.

Activation Thresholds

Provider	Threshold
Anthropic (Claude)	10,000 tokens
OpenAI (GPT-4, etc.)	10,000 tokens
Google (Gemini)	10,000 tokens
Default	10,000 tokens

Override with `--kompressor-threshold N` or `HEADROOM_KOMPRESSOR_THRESHOLD=N`. Set to `0` to disable Kompressor entirely.

Compression Process (Step by Step)

1. Scan each message's estimated token count.
2. If below threshold → skip, message passes through unchanged.
3. If above threshold → check if the message is a tool result, file read, or search output.
4. Detect the format (JSON, code, log, plain text) and choose the appropriate compressor.
5. Run the compressor, targeting a reduction ratio (default: keep ~30% of tokens).
6. Replace the message content with the compressed version.
7. Record the savings in the transform result for metrics tracking.

Smart Summary Discard

When Kompressor compresses a message, it doesn't just truncate. It **summarizes** the discarded portion — keeping the first few lines, the last few lines, and a structural summary of what was in the middle. This preserves the model's ability to understand the overall shape of the data.

Provider Restriction (Paid Only)

Kompressor is restricted to paid providers (Anthropic, OpenAI, Google) by default. Free-tier providers (Groq, Together, Ollama) are excluded because the compression overhead can actually increase latency on fast, cheap models.

Provider	Kompressor Enabled?
Anthropic	Yes — Claude models
OpenAI	Yes — GPT-4, GPT-3.5
Google	Yes — Gemini models
Groq, Together, Ollama, etc.	No — free tier

Override with `--kompressor-all-providers` or `HEADROOM_KOMPRESSOR_ALL_PROVIDERS=true`.

Result Files

Each compression result is stored in memory with:

- Original token count
- Compressed token count
- Compression ratio
- Format detected
- Compressor used

Quota & Limit Banners

Free tier — "You've used 80% of your daily limit. Upgrade to Pro for unlimited access." Dismissible, reappears next session.

Pro tier — "You've reached your monthly limit. Overage billing is active." Non-dismissible, persistent.

Context Window Exceeded Handling

When the Anthropic API returns a context_length_exceeded error, the proxy automatically:

1. **Parses the error** — Extracts the actual token count and the limit from the error message.
2. **Trims messages** — Removes the oldest non-system messages until the request fits.
3. **Preserves system messages** — Never trims system prompts or the most recent user message.
4. **Retries once** — Sends the trimmed request. If it fails again, returns the error to the client.
5. **Records the event** — Logs how many messages were dropped and how many tokens were saved.

The trimming strategy is conservative — it drops the minimum number of messages needed.

Streaming responses are handled identically — the error is caught before any SSE events are sent.

API Proxy Documentation Headers

Every response from the proxy includes custom headers that describe what happened to your request:

- **X-Headroom-Tokens-Before** — Token count before compression
- **X-Headroom-Tokens-After** — Token count after compression
- **X-Headroom-Transforms** — Comma-separated list of transforms applied

These headers are visible in browser DevTools, curl -v output, and any HTTP client that inspects response headers.

Download Mode

The dashboard supports downloading individual sessions as JSON files. Click the download button on any session in the Sessions tab.

- **Single session** — Downloads one session as a pretty-printed JSON file.
- **Filtered export** — Downloads all sessions matching the current search/filter as a JSON array.
- **Format** — Uses the same schema as `GET /sessions/{id}`.

Responsive Design

The dashboard is fully responsive and works on desktop, tablet, and mobile.

- **Desktop (≥ 1024px)** — Full layout with all columns visible. Charts render at full size.
- **Tablet (768px – 1023px)** — Compact layout. Secondary columns hidden. Charts scale down.
- **Mobile (< 768px)** — Single-column layout. Tables become scrollable. Navigation collapses into a hamburger menu.

DeepSeek V4 Migration

Background

New Models

Architecture Improvements (V4 vs. V3)

Deprecation Timeline

Changes Made in This Project (11 May 2026)

API Compatibility

---

19 July 2026 Maintenance & Feature Update

1. Critical Bug: `deepseek-v4-pro` Was Never Reachable

2. Diagnostic Logging: Model Name in Server Log

3. Dead Model Cleanup

4. OpenAI Lineup Updated to GPT-5.5 / GPT-5.6

5. Transparent API Error Messages

6. `max_tokens` → `max_completion_tokens` for OpenAI

7. Vision Pipeline Completed

8. `deploy.sh`: MD5 Checksum Verification

Files Changed

---

28 August 2026 Fix: Explicit DeepSeek Thinking Mode Control

Files Changed

Security Hardening (26 July – 15 September 2026)

Stage 1 — Stored XSS and Path Traversal

Stage 2 — CORS Wildcard and Request Size Limits

Stage 3 — Log Exposure and Host Header Injection

Stage 4 — Deliberately Not Implemented: Endpoint Authentication

Stage 5 — Documentation Gap and Deferred Modularization

Residual XSS Follow-Up (14 September 2026)

HTTPS Migration: From Self-Signed to Let's Encrypt

The Fix

A Naming Trap, Caught Before Deployment

Result

Side Effects Found Along the Way

---

17 September 2026: Model Display Labels & Cleanup

Removed: `deepseek-v4-flash-vision-exp`

Added: Readable Model Labels with Release Dates

A Bug in the First Pass

The Helper Script `repo2text.sh`

Security Architecture in Detail

1. API Keys — Never Exposed to the Client

2. Magic Byte Inspection

3. Secure Session Storage

4. Log Without Sensitive Data

5. No Direct Client-API Communication

6. Input Validation

7. Transport Security

Deployment & Usage

Prerequisites

Installation

4. Create a new API Environment

```
npm run dev:environment -- --name "Staging" --base-url "https://staging.api.example.com"
```

5. Register a new route

```
npm run dev:route -- --path "/v2/users" --method "GET" --handler "src/handlers/users.list"
```

6. Run the full test suite

```
npm test -- --coverage --reporter=verbose --bail --timeout=30000
```

Configuration

Environment Configuration Create a `.env.local` file in the root of the project. Never commit this file. It should already be listed in `.gitignore`.

```
# ─── Application ──────────────────────────────────────────────
APP_NAME=MyProject
APP_ENV=development
APP_PORT=3000
APP_SECRET=change_me_to_a_long_random_string
APP_LOG_LEVEL=debug

# ─── Database ─────────────────────────────────────────────────
DB_HOST=localhost
DB_PORT=5432
DB_NAME=myproject_dev
DB_USER=myproject_user
DB_PASSWORD=change_me
DB_POOL_MIN=2
DB_POOL_MAX=10
DB_SSL=false

# ─── Redis ────────────────────────────────────────────────────
REDIS_URL=redis://localhost:6379
REDIS_PASSWORD=
REDIS_TLS=false

# ─── Auth ─────────────────────────────────────────────────────
JWT_ACCESS_SECRET=change_me_access_secret
JWT_REFRESH_SECRET=change_me_refresh_secret
JWT_ACCESS_EXPIRES_IN=15m
JWT_REFRESH_EXPIRES_IN=7d
OAUTH_GOOGLE_CLIENT_ID=
OAUTH_GOOGLE_CLIENT_SECRET=
OAUTH_GITHUB_CLIENT_ID=
OAUTH_GITHUB_CLIENT_SECRET=

# ─── Email ────────────────────────────────────────────────────
SMTP_HOST=smtp.mailtrap.io
SMTP_PORT=2525
SMTP_USER=
SMTP_PASSWORD=
EMAIL_FROM="MyProject <noreply@example.com>"

# ─── Storage ──────────────────────────────────────────────────
STORAGE_DRIVER=local
STORAGE_LOCAL_PATH=./uploads
STORAGE_S3_BUCKET=
STORAGE_S3_REGION=
STORAGE_S3_ACCESS_KEY=
STORAGE_S3_SECRET_KEY=

# ─── Feature Flags ────────────────────────────────────────────
FEATURE_SIGNUP_ENABLED=true
FEATURE_OAUTH_ENABLED=false
FEATURE_MAINTENANCE_MODE=false

# ─── Rate Limiting ────────────────────────────────────────────
RATE_LIMIT_WINDOW_MS=60000
RATE_LIMIT_MAX_REQUESTS=100
```

Never commit `.env.local` or any file containing real credentials. Add it to `.gitignore` immediately if it isn't already there.

Configuration Loading Environment variables are loaded at startup in `src/config/index.ts` using `dotenv`. All values are validated against a schema (`src/config/schema.ts`) — the app will refuse to start if required variables are missing or malformed.

Per-Environment Overrides In addition to `.env.local`, you can create `.env.staging` and `.env.production` files for environment-specific defaults. These are also gitignored. Load order: `.env.local` > `.env.[APP_ENV]` > `.env`.

Deploy Scripts

Script	Purpose
`./deploy.sh staging`	Builds the Docker image, pushes it to the registry, and deploys to the staging environment. Requires `STAGING_HOST` and `DEPLOY_KEY` to be set.
`./deploy.sh production`	Same as staging but targets production. Requires an explicit confirmation prompt.
`./rollback.sh`	Reverts to the previous deployment. Uses the last known good image tag stored in `.deploy_history`.
`./migrate.sh up`	Runs all pending database migrations against the target environment.
`./migrate.sh down`	Rolls back the most recent migration. Prompts for confirmation before proceeding.
`./seed.sh`	Populates the database with development fixtures. Refuses to run against production.

Important: Always run **`./migrate.sh up`** before deploying new code that depends on schema changes. Migrations are forward-compatible by convention — we avoid dropping columns in the same release that stops using them. See `docs/migrations.md` for the full policy.

```
# Example: deploy to staging after running tests locally
npm test && ./deploy.sh staging
```

The script will print the deployment ID and tag when it finishes. If anything looks wrong, run `./rollback.sh` immediately — it's faster than debugging a broken deploy in production.

Project Structure

```
.
├── .github/
│   ├── workflows/
│   │   ├── ci.yml              # Runs tests on every push
│   │   └── deploy.yml          # Deploys on merge to main
│   └── CODEOWNERS
├── .husky/
│   ├── pre-commit              # Runs lint-staged
│   └── commit-msg              # Enforces conventional commits
├── .vscode/
│   ├── extensions.json         # Recommended extensions
│   └── settings.json           # Workspace editor settings
├── docs/
│   ├── adr/                    # Architecture Decision Records
│   ├── api.md                  # API reference
│   └── migrations.md           # Migration policy and guidelines
├── infra/
│   ├── docker-compose.yml      # Local development services
│   ├── docker-compose.test.yml # Test database and services
│   └── terraform/              # Infrastructure as code
├── scripts/
│   ├── deploy.sh               # Deployment script
│   ├── migrate.sh              # Database migration runner
│   ├── rollback.sh             # Rollback to previous version
│   ├── seed.sh                 # Seed development data
│   └── setup.sh                # First-time project setup
├── src/
│   ├── config/
│   │   ├── index.ts            # Loads and validates env vars
│   │   └── schema.ts           # Config validation schema
│   ├── db/
│   │   ├── migrations/         # Database migration files
│   │   ├── seeds/              # Development seed data
│   │   └── client.ts           # Database connection pool
│   ├── handlers/               # Request handlers (controllers)
│   ├── middleware/
│   │   ├── auth.ts             # Authentication middleware
│   │   ├── error.ts            # Global error handler
│   │   ├── logger.ts           # Request logging
│   │   └── rate-limit.ts       # Rate limiting
│   ├── models/                 # Data models
│   ├── routes/
│   │   ├── index.ts            # Route registration
│   │   └── v1/                 # Version 1 routes
│   ├── services/               # Business logic
│   ├── utils/                  # Shared utilities
│   └── app.ts                  # Application entry point
├── tests/
│   ├── fixtures/               # Test data
│   ├── helpers/                # Test utilities
│   ├── integration/            # Integration tests
│   └── unit/                   # Unit tests
├── .dockerignore
├── .env.example
├── .gitignore
├── .nvmrc
├── Dockerfile
├── Makefile
├── package.json
└── README.md
```

Model Configuration

The `model_config.json` file is the **single point of truth** for all model-specific behavior. Every component that touches model inputs — tokenization, chunking, prompt formatting, inference parameters, output parsing — reads from this file. No model-specific values are hardcoded anywhere else in the codebase.

Switching model configurations When you switch models, validate that the new file's values match the model's actual specifications. Mismatched `max_input_tokens` or `chunk_overlap_tokens` will cause silent truncation or degraded retrieval quality rather than explicit errors.

Example configurations for [Llama 3.1 8B Instruct](), [Llama 3.1 70B Instruct](), [Mistral 7B Instruct v0.3](), and [Qwen 2.5 7B Instruct]() are provided in the `examples/` directory. Each has been tested against the corresponding model version listed. If you're using a different quantization (e.g., GPTQ, AWQ, or a different GGUF quant level), you may need to adjust `max_input_tokens` downward to account for reduced context fidelity — see the model's documentation for guidance.

Design Manifest

The project follows a set of design principles that guide architectural decisions and trade-offs. These are:

Offline-first. All core functionality must work without network access. Any feature that requires connectivity must degrade gracefully when offline.

Explicit over implicit. Configuration, dependencies, and behavior should be visible and predictable. Avoid magic that makes it hard to understand what's happening.

Small surface area. Every public API, CLI flag, and configuration option is a commitment. Add them deliberately and remove them aggressively.

Data ownership. Users own their data. Export should be easy, lossless, and well-documented.

Boring technology. Prefer well-understood tools and patterns over novel ones unless the novel approach offers a clear, significant advantage.

Progressive disclosure. Simple tasks should be simple. Complexity should only appear when the user asks for it.

Fail loudly, recover quietly. Surface errors immediately during development. In production, retry transient failures automatically but never silently swallow errors.

Accessibility is not optional. All UI must meet WCAG 2.1 AA standards. Keyboard navigation, screen reader support, and sufficient color contrast are requirements, not nice-to-haves. See `docs/accessibility.md` for our testing checklist.

Known Limitations & Technical Notes

"Lost in the Middle" — A Known AI Limitation

GitHub Raw URL Caching

Nano and Unicode — Critical Warning

Linux/X11/Firefox Paste Behavior

Context Overflow Detection Edge Cases

DeepSeek Model Self-Reporting

Dependencies

Conclusion / Why This Project Stands Out